PORTFOLIO CASE STUDY

TaskFlow

A role-based task scheduling & team-tracking platform, evolved from a classroom Kanban prototype into a multi-role product foundation for real company use.

React 18 + MUI | Flask 3 + Python | MongoDB | JWT Auth (access + refresh) | Role-Based Access Control | Drag & Drop Kanban | Docker Compose | Pytest

---

Prepared August 2026

Full-stack rebuild: data model, auth, API, and UI

Contents

01 Core Idea & Problem Statement

02 System Architecture

03 Data Model

04 Key Algorithms & Logic

05 Application Walkthrough (Screenshots)

06 Security Posture

07 Suggestions for Future Development

01 · Core Idea & Problem Statement

TaskFlow began as a four-column Kanban board built for a university group project — a single Flask file writing one unstructured MongoDB document, with no accounts, no ownership of tasks, and no way to tell who was doing what. The rebuild reframes the same domain (todo / doing / done / dropped tasks) as a **role-based task-scheduling product a company could actually deploy**: managers plan and assign work with due dates, employees execute and report progress, and both sides see the same board through a lens shaped by their role.

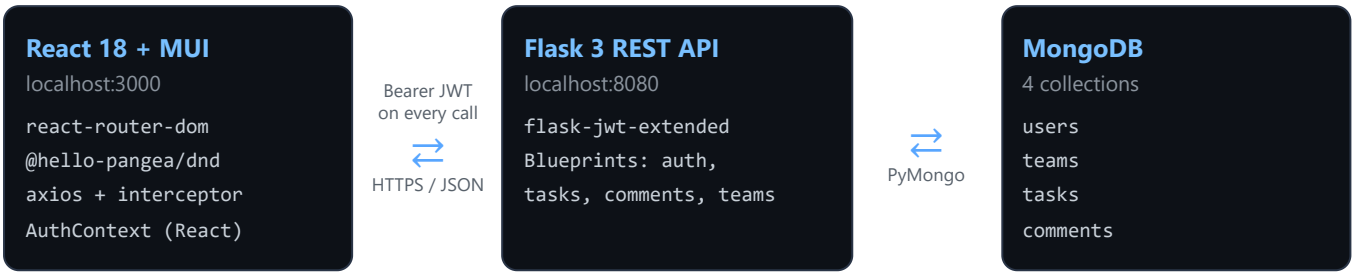
> **DESIGN PREMISE**
>
> The product optimizes for three actors interacting with one shared board: a **manager** who schedules and assigns future work, an **employee** who executes it and narrates progress, and the **team** as a whole, whose aggregate state a manager needs to see at a glance without opening every task.

What changed from prototype to product

Prototype (original)	TaskFlow (this build)
No accounts — anyone with the URL can edit the board	Manager/employee accounts, JWT-authenticated, scoped to a team
One Mongo document holding all tasks as string arrays	Four normalized collections (`users` , `teams` , `tasks` , `comments`) with real relationships
Tasks have a title only — no owner, due date, or history	Tasks carry assignee, priority, due date, status, and a comment/progress-report feed
"Move task" via a popup with hard-coded adjacent-column buttons	True drag-and-drop board plus a full edit dialog, permission-gated per field
Every page load overwrote the database with an empty object (a live bug)	Proper REST API; state is fetched, never blindly overwritten

02 · System Architecture

The system is a conventional three-tier web app, deliberately kept simple to match the size of the problem — no microservices, no message queue, nothing the team doesn't need yet.



Backend layout

The Flask app is built with an application-factory pattern (`create_app()`) rather than the prototype's single global-scope script. Each resource gets its own blueprint, so route logic, permission checks, and Mongo queries live next to each other instead of sprawling across one file:

```
backend/
  app.py            → create_app(): registers blueprints, CORS, JWT
  config.py         → env-driven config (MONGO_URI, JWT secret, expiries)
  extensions.py     → shared PyMongo + JWTManager singletons
  routes/
    auth.py         → register-manager / register-employee / login / refresh / me
    tasks.py        → task CRUD, all permission logic
    comments.py     → per-task comment thread
    teams.py        → team member list + manager dashboard aggregation
  utils/
    ids.py          → safe ObjectId parsing
    auth.py          → role_required() decorator
  tests/            → 24 pytest cases over auth + permissions
```

Frontend layout

```
frontend/src/
  api/client.js          → axios instance, token attach + silent-refresh interceptor
  context/AuthContext.js → session state, login/register/logout
  components/
    ProtectedRoute.js, RoleGate-style guards
    TaskCard.js, TaskFormDialog.js, CommentFeed.js, Navbar.js
  pages/
    LoginPage, RegisterPage, BoardPage, TeamDashboardPage
  App.js                 → React Router routes + MUI dark theme
```

WHY MONGODB, NOT A RELATIONAL DB

A relational schema was considered, since users/tasks/comments are naturally foreign-keyed — but the team's own MongoDB instance was a hard requirement, and Mongo handles this shape fine once split into real collections (rather than the prototype's single blob document). Every collection carries a `team_id`, which is what makes multi-team data isolation possible without a relational join.

03 · Data Model

Four collections replace the prototype's single unstructured document. Every write is validated server-side; the client never dictates a task's `team_id` or `created_by` — those are derived from the verified JWT, not the request body.

`users`

```
{
  _id, name, email, password_hash,   // PBKDF2-SHA256 via werkzeug.security
  role: "manager" | "employee",
  team_id, created_at
}
```

`teams`

```
{
  _id, name, manager_id,
  join_code,     // 8-char hex, e.g. "DF677255" — how employees attach to a team
  created_at
}
```

`tasks`

```
{
  _id, title, description,
  status: "todo" | "doing" | "done" | "dropped",
  priority: "low" | "med" | "high",
  start_date, due_date,             // ISO date strings
  assignee_id, created_by, team_id,
  created_at, updated_at
}
```

`comments`

```
{
  _id, task_id, author_id, body, created_at
}
```

Comments deliberately serve two purposes at once — casual discussion *and* employee progress reports — rather than building a separate "status report" subsystem. A comment box on every task is enough for "report their working process," per the product's brief, without inventing a second, parallel feed that would just duplicate the first.

04 · Key Algorithms & Logic

4.1 — Role-based field permissions on a single endpoint

Rather than building separate manager-edit and employee-edit endpoints, `PUT /tasks/<id>` builds its update payload conditionally based on the caller's role, resolved from JWT claims — never from the request body:

```
is_manager   = claims["role"] == "manager"
is_assignee  = task["assignee_id"] == current_user_id

if is_manager:
    # title, description, priority, dates, assignee are all eligible
    updates["title"] = ...  # etc., only if present in the request

if "status" in request_body:
    # status is the one field BOTH a manager and the task's own
    # assignee are allowed to change
    updates["status"] = request_body["status"]

if not updates:
    return 400  # nothing this caller is allowed to change was sent
```

An employee sending `{"title": "...", "status": "doing"}` gets a 200 with the title silently ignored and the status applied — proven by a dedicated pytest case. An employee sending only `{"title": "..."}` gets a 400, since nothing in the payload was theirs to change. This single conditional-build pattern is what lets one route safely serve two roles with very different privileges.

4.2 — Team isolation via JWT-embedded claims

Every list/read/write query is filtered by `team_id`, and that value is read from the verified JWT's claims, never from a query parameter or JSON body:

```
claims = get_jwt()
team_id = oid(claims["team_id"])
tasks = mongo.db.tasks.find({"team_id": team_id, ...})
```

This closes off an entire class of bug: a client cannot simply pass a different `team_id` to read another company's board, because the server never looks at a client-supplied team id at all. A pytest case (`test_tasks_are_scoped_to_team`) proves a second manager's `GET /tasks` returns an empty list even though tasks exist — just for a different team.

4.3 — Join-code based team onboarding

Rather than an invite-email system (out of scope for this pass), a manager's `register-manager` call creates a team and mints an 8-character code via `secrets.token_hex(4).upper()` — cryptographically random, short enough to read aloud, long enough ($16^8 \approx 4.3$ billion combinations) to resist guessing. An employee's registration simply looks up a team by that code; no code, no team, no way to attach yourself to a company you don't have the code for.

4.4 — Access + refresh token rotation (silent re-auth)

Access tokens are intentionally short-lived (15 minutes) to limit the blast radius of a leaked token, while a 30-day refresh token lets a user stay logged in without re-entering a password. The interesting part is the client-side interceptor that makes the trade-off invisible to the user:

```
// axios response interceptor, simplified
on 401 response:
  if request was already retried, or hit an auth endpoint → give up
  if a refresh is already in flight → await that shared promise
    (prevents 5 simultaneous 401s from firing 5 refresh calls)
  else → POST /auth/refresh with the refresh token, store new access token
  retry the original request once, marked _retry, with the new token
  if refresh itself fails → clear tokens, broadcast "auth:logout" →
    AuthContext hears it, sets user = null, ProtectedRoute redirects
```

This was verified end-to-end by minting a deliberately-expired access token and confirming the network trace: `GET /auth/me → 401 → POST /auth/refresh → 200 → GET /auth/me (retried) → 200`, with the user never seeing a login screen.

4.5 — Manager dashboard aggregation & overdue detection

`GET /teams/dashboard` computes, per team member, a live rollup rather than maintaining a denormalized counter that could drift out of sync:

```
for member in team_members:
    member_tasks = [t for t in all_team_tasks if t.assignee_id == member._id]
    overdue = count of member_tasks where:
        due_date exists
        AND status not in (done, dropped)
        AND due_date < now (UTC)
    counts = { todo, doing, done, dropped } via simple status tally
    completion_pct = done_count / total_count
```

Computing this at request time (rather than via a background job) keeps it always correct with zero staleness risk, at the cost of O(tasks × members) work per dashboard load — a deliberate trade-off that's free at this scale and would only need revisiting if a team grew into the thousands of tasks.

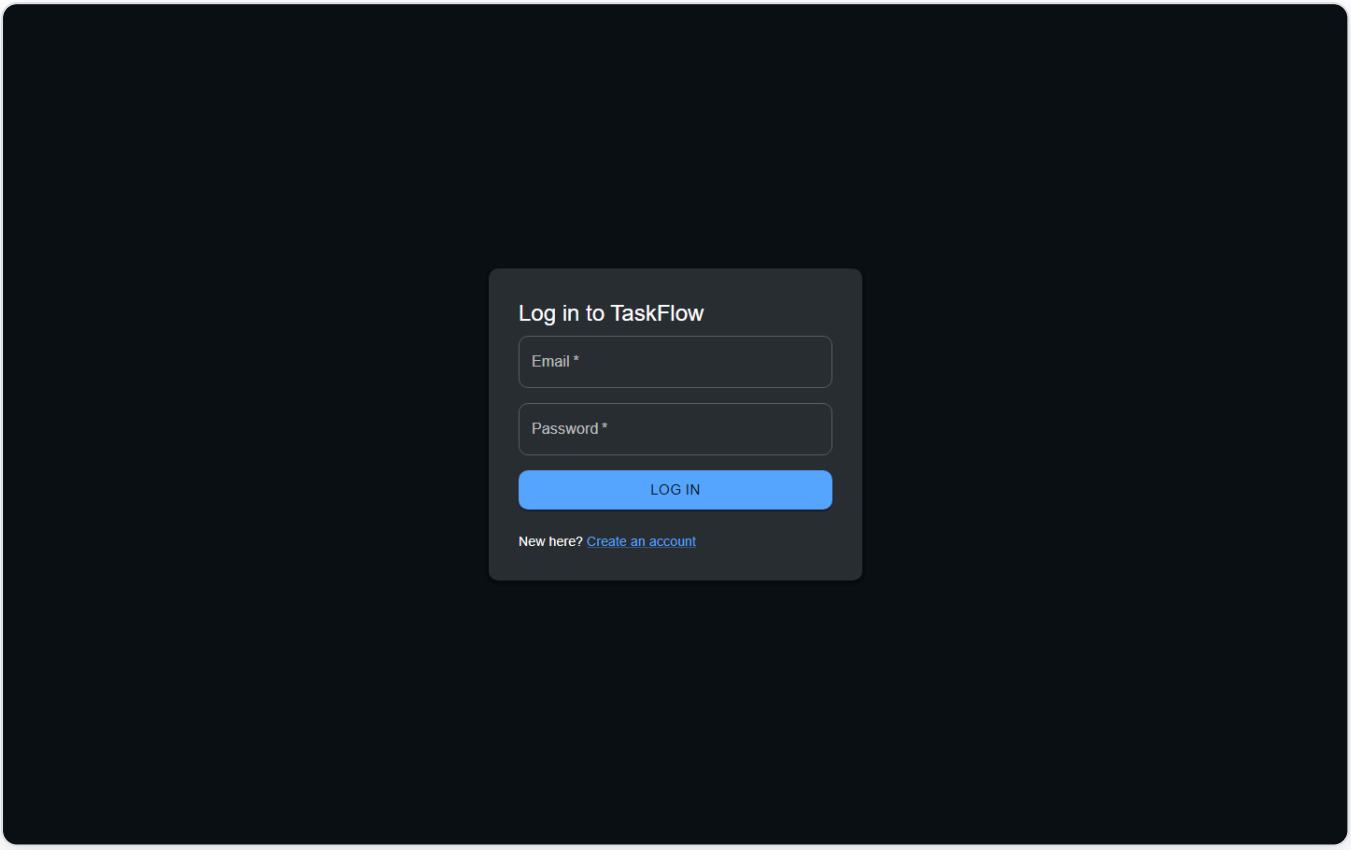
4.6 — Drag-and-drop as a status-only state transition

The board uses `@hello-pangea/dnd`'s `DragDropContext`, with each of the four columns as a `Droppable` and each card as a `Draggable`. Dropping a card in a new column optimistically updates local state, then fires `PATCH /tasks/<id>/status`; if the request fails, the board silently reloads from the server to self-correct. Drag is disabled per-card (`isDragDisabled`) unless the current user is the manager or that task's own assignee — the same authorization rule as the edit dialog, just expressed as a UI affordance instead of a form field.

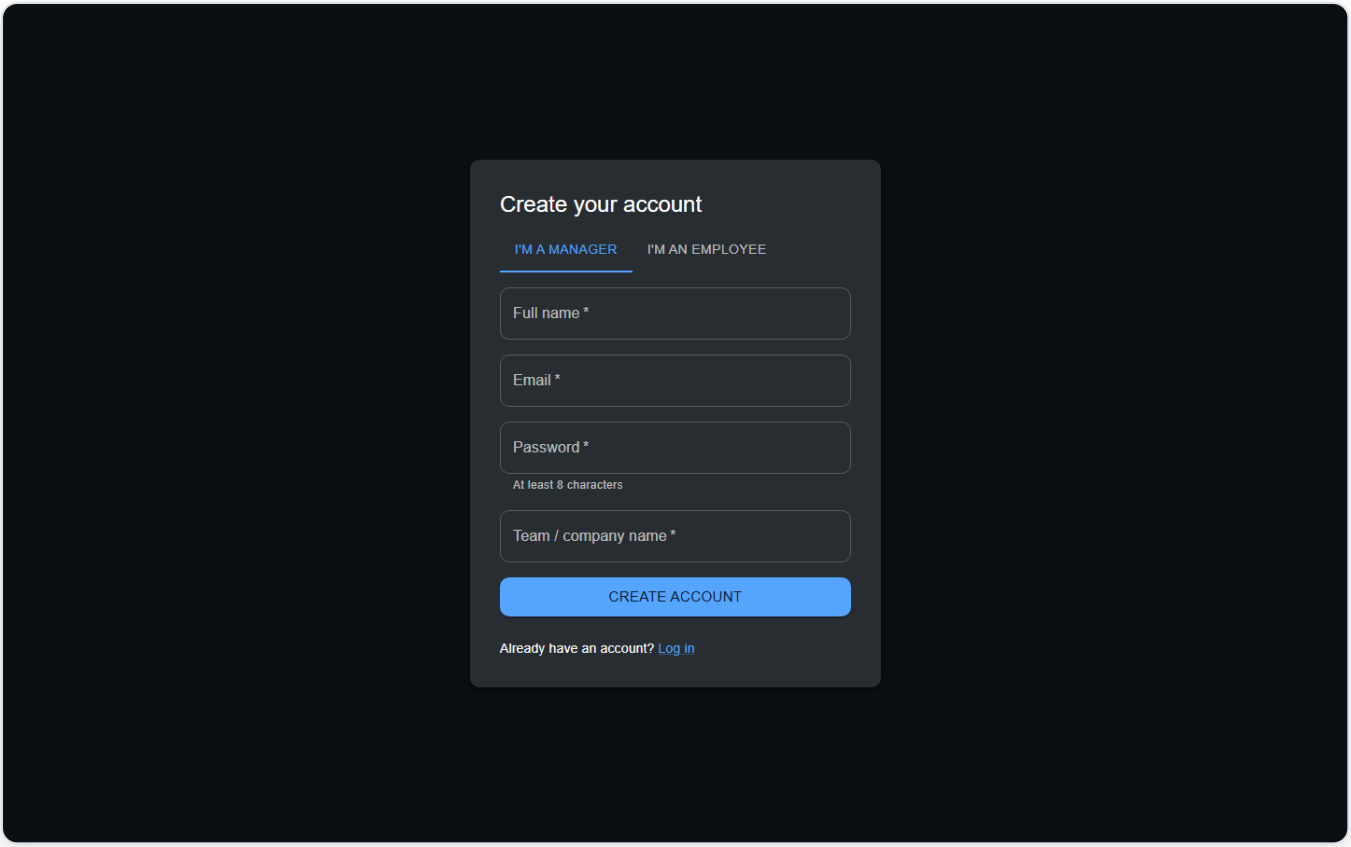
05 · Application Walkthrough

The following sequence traces one full session end-to-end: a manager creates a team and a task, an employee joins and works the task, and the manager checks progress before logging out — the exact loop TaskFlow is built around.

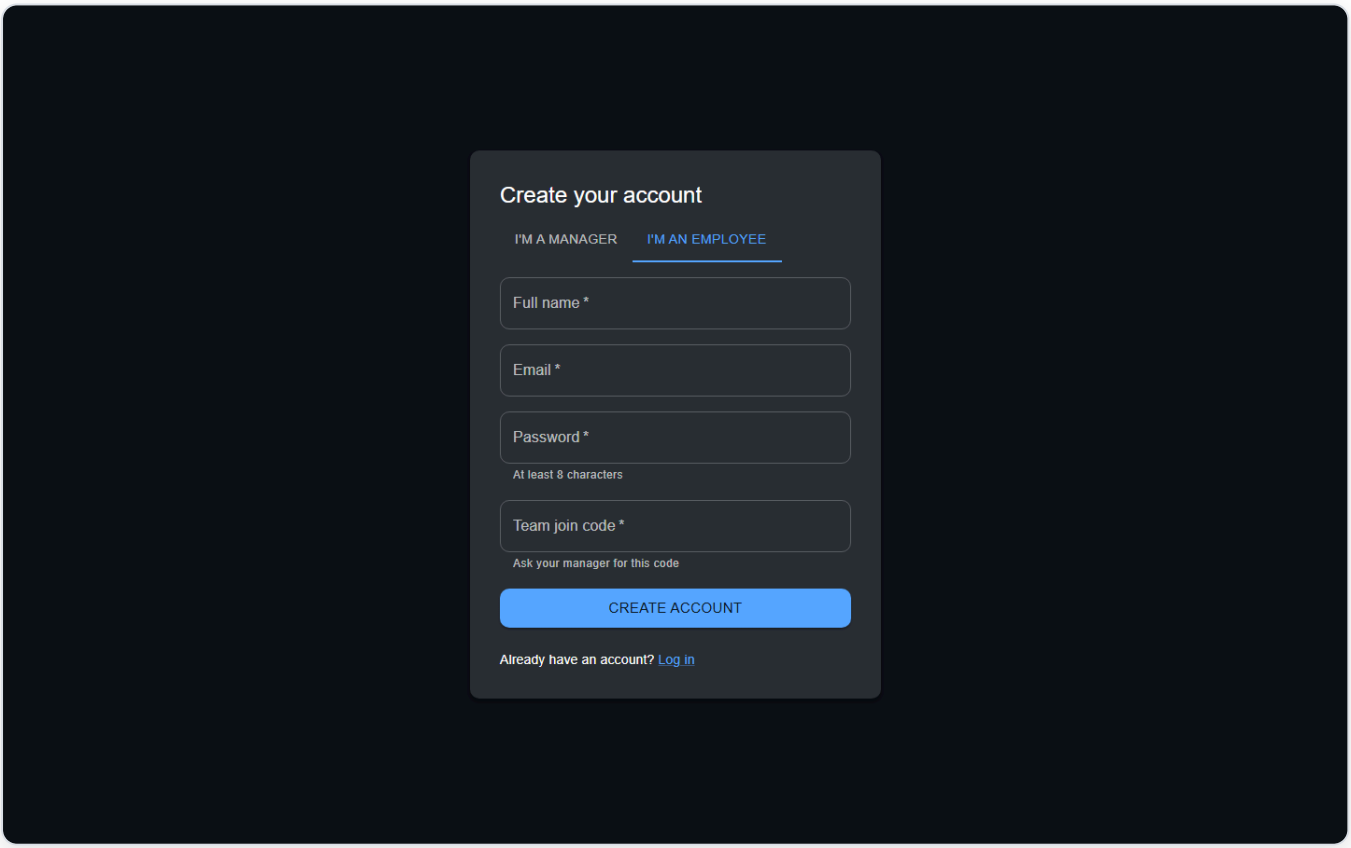
1 **Login** — returning users authenticate with email + password. A JWT access/refresh pair is issued and the session persists silently via the refresh interceptor described in §4.4.



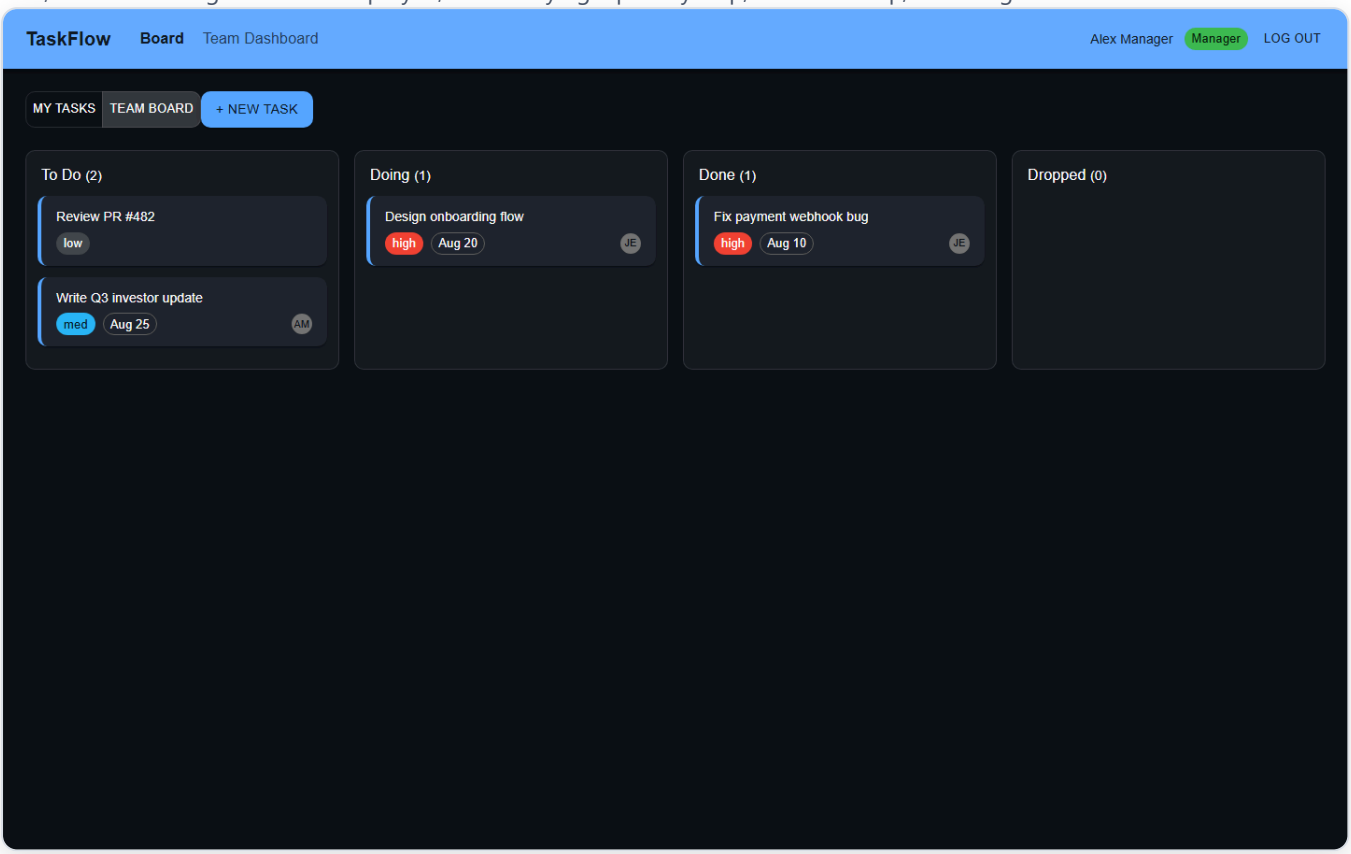
2 **Register — manager path** — a new manager names their team; the server creates the team and hands back a join code shown once on screen (and copyable from a future team-settings page).



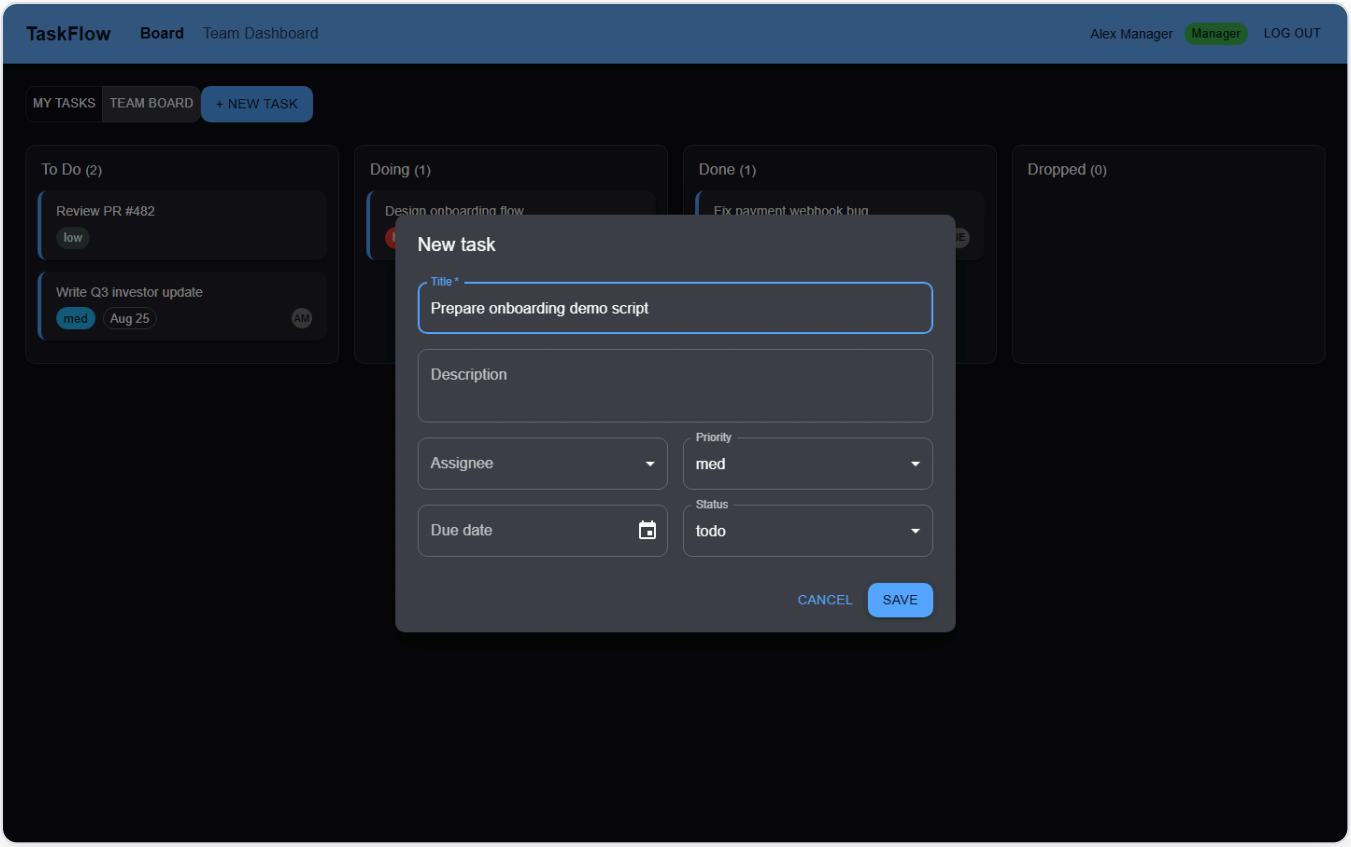
3 **Register — employee path** — the same form adapts to a join-code field instead of a team-name field, matching the two onboarding paths described in §4.3.



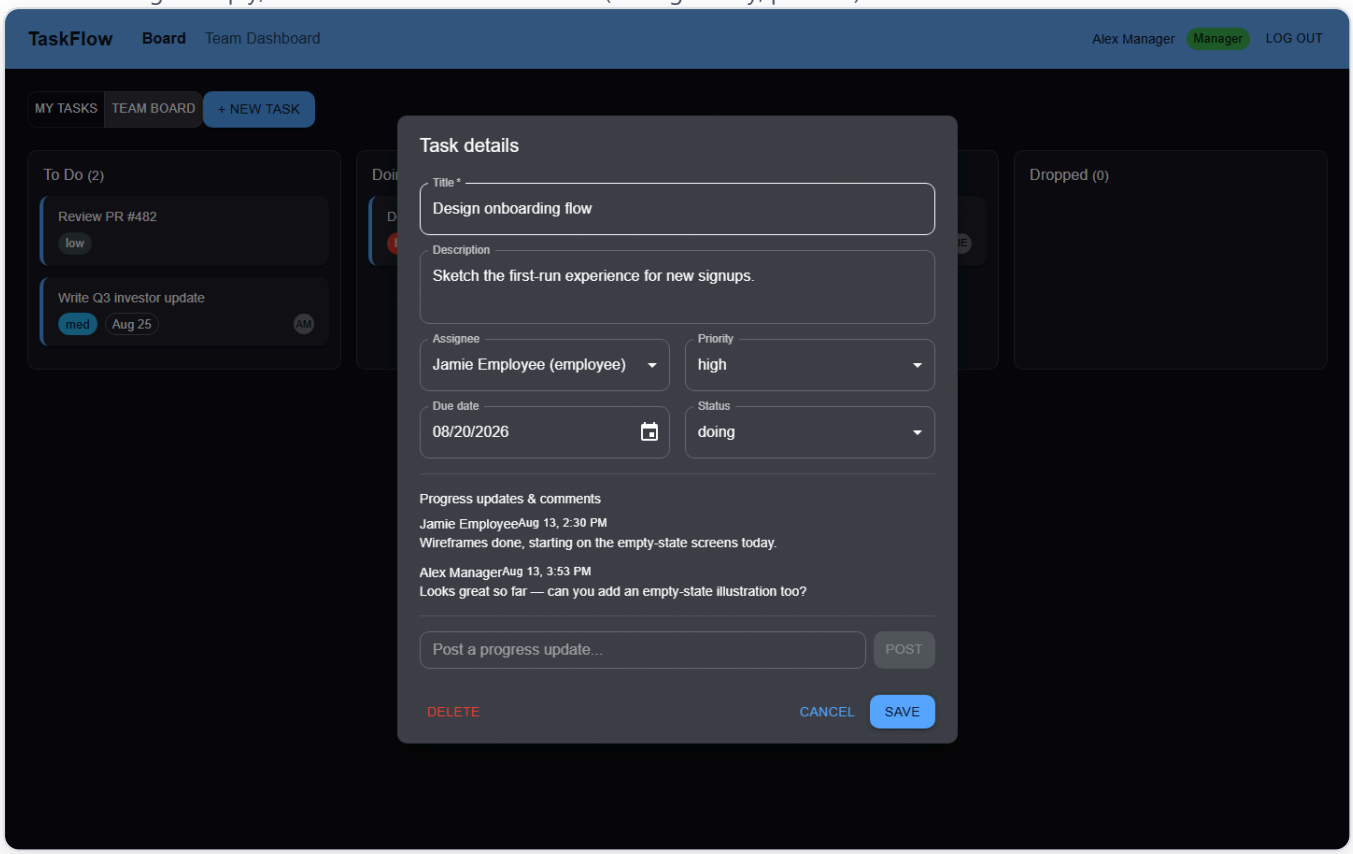
4 **Team Board (manager view)** — four columns, populated with a realistic spread of tasks: unassigned work, a self-assigned task, and work delegated to an employee, each carrying a priority chip, due-date chip, and assignee avatar.



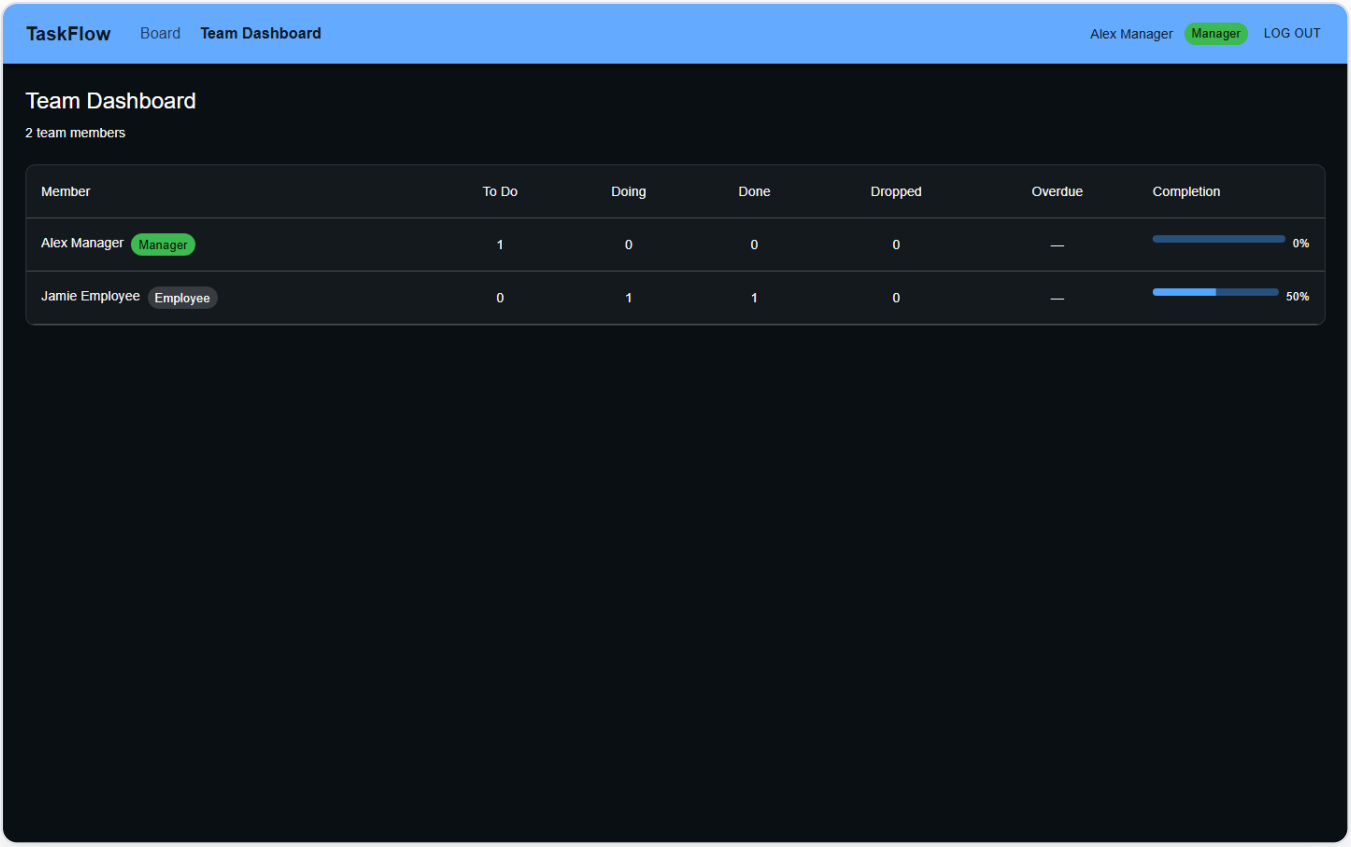
5 **Creating a task** — only visible to managers. Title, description, assignee (drawn live from the team roster), priority, due date, and initial status are all captured in one MUI dialog.



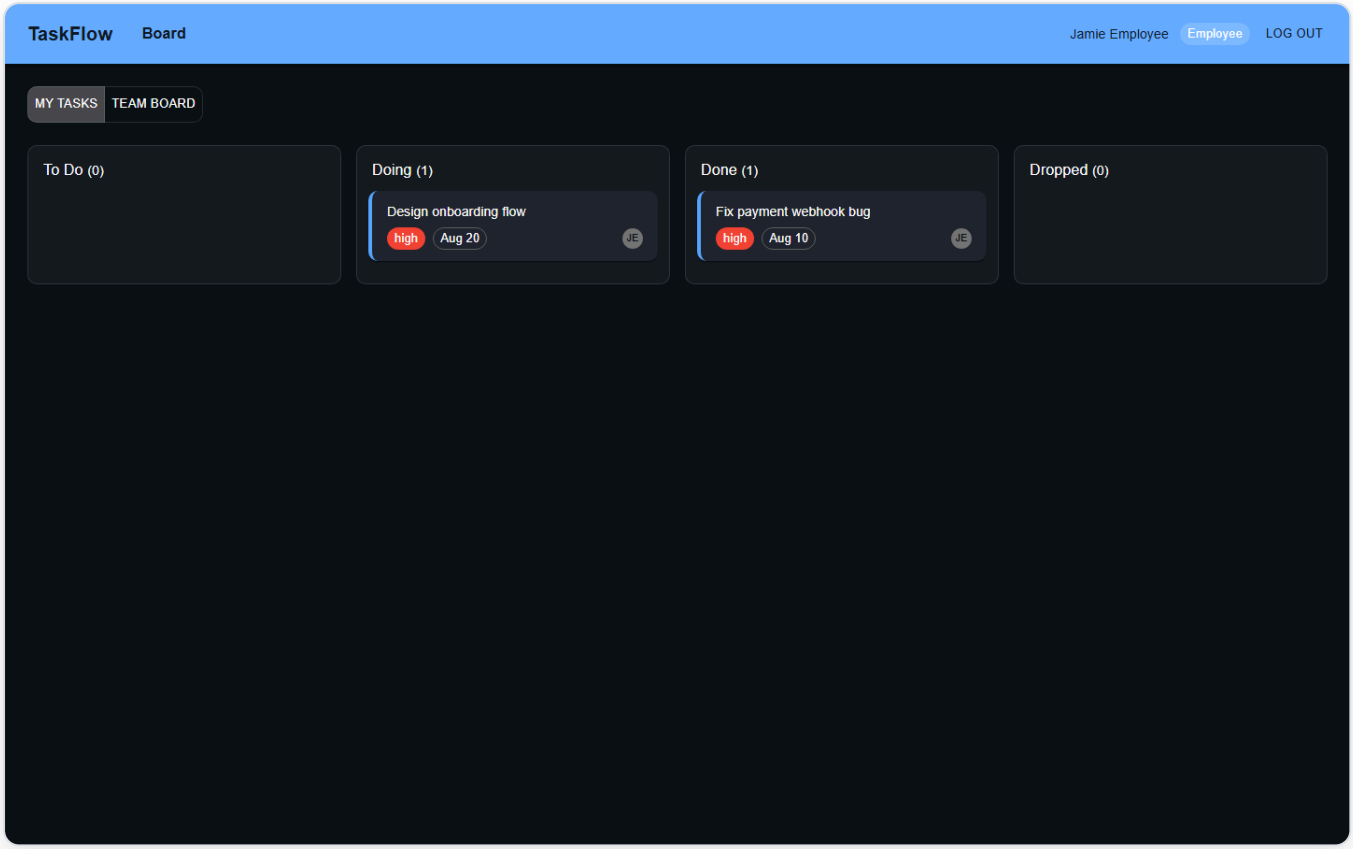
6 **Task detail (manager view)** — every field is editable, the comment thread shows both the employee's progress update and the manager's reply, and a Delete action is available (manager-only, per §4.1).



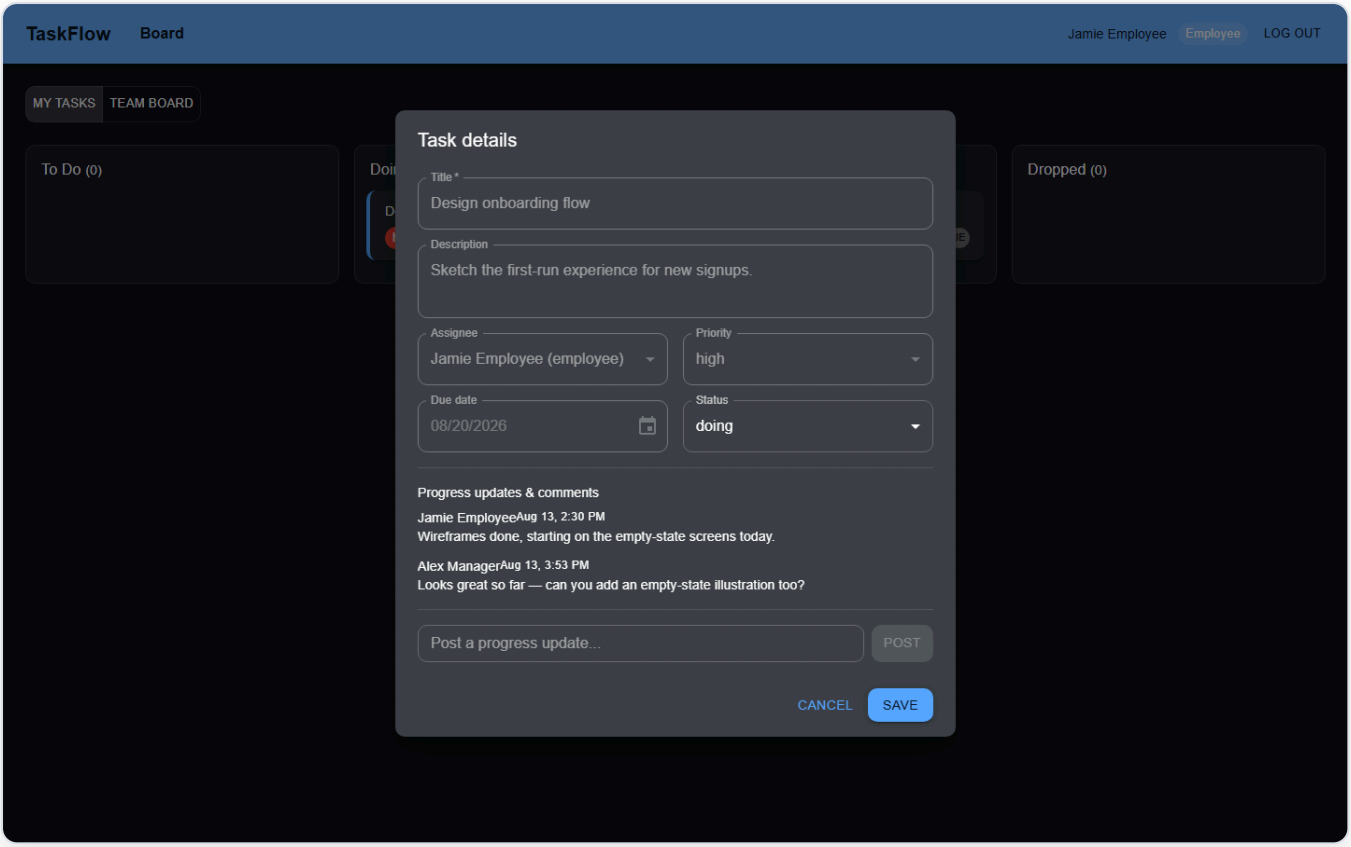
7 **Team Dashboard** — the live aggregation from §4.5: per-member status counts, an overdue flag, and a completion-percentage bar the manager can scan without opening a single task.



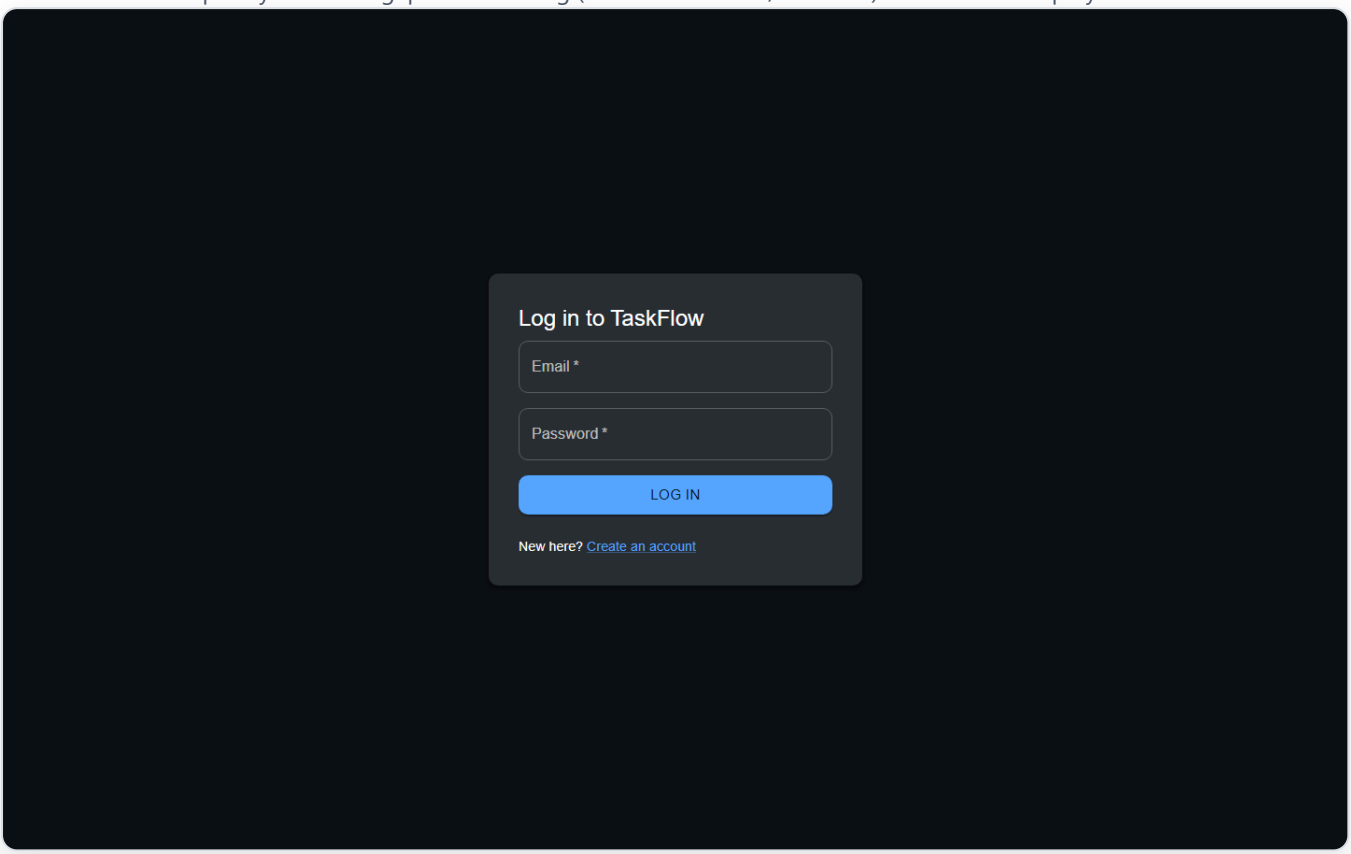
8 **Board (employee view — "My Tasks")** — the same board component, scoped by the `scope=mine` query param, so an employee's default view is just their own workload.



9 **Task detail (employee view)** — title, description, assignee, priority, and due date render read-only; only Status and the comment box are interactive, and there is no Delete action — the exact permission boundary proven by the pytest suite in §4.1.



10 **Logout** — clears both tokens client-side and returns to the login screen. No server-side session to invalidate exists yet; this is called out explicitly in §7 as a gap worth closing (token revocation / blocklist) before a real deployment.



06 · Security Posture

Concern	Status
Password storage	Hashed with werkzeug's PBKDF2-SHA256 (`generate_password_hash`) — never stored or logged in plaintext
Authentication	JWT access + refresh tokens (flask-jwt-extended); access tokens expire in 15 minutes
Authorization	Enforced server-side on every route via role claims + assignee checks — never trusted from the client
Cross-tenant data isolation	Every query scoped by JWT-derived `team_id` ; verified by automated tests
Secrets management	Mongo URI / JWT secret loaded from a gitignored `.env` , never hard-coded (a real issue in the original prototype)
Input validation	Status/priority values are allow-listed server-side; empty titles, short passwords, and malformed IDs are rejected with 400s
Not yet implemented	Rate limiting on auth endpoints; token revocation/blocklist on logout; password reset flow; email verification

07 · Suggestions for Future Development

Near-term (closes real gaps before pilot use)

- **Password reset / forgot password** — currently no recovery path exists; a real blocker for any company actually adopting this.
- **Rate limiting** on `/auth/login` and `/auth/register-*` to blunt brute-force attempts.
- **Join-code recovery** — the code is only ever shown once at team creation; a manager who loses it currently has no way to retrieve or rotate it.
- **Refresh token revocation** — logout only clears client-side storage today; a stolen refresh token remains valid until it naturally expires. A server-side denylist (or short-lived refresh + rotation) would close this.

Product depth

- **Notifications** — email or in-app alerts on assignment, comments, and approaching due dates, turning the "scheduling" half of the product from passive to active.
- **Calendar / timeline view** — a due-date-ordered view across the whole team, complementing the kanban board rather than replacing it.
- **Manager analytics & export** — completion trends over time, CSV/PDF reports, the kind of artifact a manager would actually show upward.
- **Team management UI** — no current way to deactivate an employee or change a role after signup; needed before this could be trusted with real headcount changes.
- **Per-task activity log** — distinct from comments, a structured "status changed / reassigned / edited" trail is the most direct answer to "track employees' working process," the product's own stated goal.

Scale & polish

- **True multi-tenant SaaS** — today each company is expected to run its own deployment/database; serving many companies from one deployment would need tenant-scoped auth throughout (the `team_id`-on-every-document pattern already in place makes this a extension, not a redesign).
- **File attachments** on tasks and comments.
- **Search & filtering** on the board by assignee, priority, or overdue status — today it's only a My Tasks / Team Board toggle.
- **Mobile-optimized layout** — the board scrolls horizontally on narrow screens but isn't touch-tuned.
- **CI pipeline** running the existing 24-case pytest suite (and a frontend test suite, currently absent) on every change.

> **WHERE I'D START**
>
> Password reset and refresh-token revocation first — they're the smallest changes with the highest downside if skipped, since nobody adopts a tool that can lock them out or that can't truly log them out.